```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();
var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit/{id}", (string id) =>
    _fruit.TryGetValue(id, out var fruit)
        ? TypedResults.Ok(fruit)
        : Results.Problem(statusCode: 404))
    .AddEndpointFilter<IdValidationFilter>();          ❶

app.Run();

class IdValidationFilter : IEndpointFilter             ❷
{
    public async ValueTask<object?> InvokeAsync(       ❸
        EndpointFilterInvocationContext context,       ❸
        EndpointFilterDelegate next)                   ❸
    {
        var id = context.GetArgument<string>(0);
        if (string.IsNullOrEmpty(id) || !id.StartsWith('f'))
        {
            return Results.ValidationProblem(
                new Dictionary<string, string[]>
                {
                    {"id", new[]{"Invalid format. Id must start with 'f'"}}
                });
        }

        return await next(context);
    }
}
```

❶ Adds the filter using the generic AddEndpointFilter method

❷ The filter must implement IEndpointFilter . . .

❸ . . . which requires implementing a single method.

Implementing `IEndpointFilter` is a good option when your filters become more complex, but note that there's no equivalent interface for the filter-factory pattern shown in section 5.4.3. If you want to generalize your filters with a filter factory, you'll have to stick to the lambda (or helper-method) approach shown in listing 5.11.